

Solar System Simulation Report

Nuno Zhan S2726638

6 April 2026

1 Introduction

The study of gravitational interactions between celestial bodies has spanned centuries, revealing dynamics far more complex than simple pairwise attraction. The solar system is a classic example of a chaotic N-body system, where long-term stability is highly sensitive to numerical precision of computational programming [3]. In this project, I develop a multi-body numerical simulation to investigate the dynamical stability and energy conservation of planetary orbits. The primary objective is to evaluate the performance of the **Beeman** integration algorithm when applied to these complex systems over decadal and centennial scales. Furthermore, I explore the occurrence of planetary alignment events, defining the criteria for such configurations based on the relative angular positions of the simulated bodies.

2 Methods

2.1 Program Design and Modularity

In this work, I mainly use three classes to perform the chaotic N-body simulation of the solar system. The **Beeman** class serves as the physics engine for individual celestial bodies by encapsulating the logic for the **Beeman** integration scheme. It stores their physical attributes such as mass, position, and velocity, as well as the energy calculation of the system. The **Simulation** class coordinates system updates. This will bridge the gap between the physical properties of the planets and the numerical integration loop. Finally, the **Animator** class provides a graphical representation of the system using **FuncAnimation**.

To optimize performance, the physics integration is independent from the visualization: the **Simulation** class first executes a `run_full_simulation` method to compute and store the entire trajectory data set. Once the state history is complete, the **Animator** reads these pre-calculated positions to render the animation. This approach ensures that the visual frame rate remains independent of the computational complexity of the N-body interactions. Additionally, to maintain the numerical stability of the whole simulation without having enormous numbers which will cause a difficulty in data interpretation, the unit of the work is set to be in **Astronomical Units (AU)** for distance, **Earth Masses** for mass, and **Earth Years (EY)** for time. The gravitational constant G is converted from SI units to simulation units:

$$G_{\text{new}} = G_{SI} \times \frac{M_{\text{Earth}}}{AU^3} \times EY^2 \quad (1)$$

which gives $G_{\text{new}} = 0.000118569$.

To establish a baseline for the Beeman algorithm's precision, I compare it against the first-order Direct Euler method and the semi-implicit Euler-Cromer method. These classes are defined using inheritance as they share foundational logic with **Beeman** class. Finally, system configurations such as the gravitational constant G , body masses, temporal settings and orbital radius, are stored in JSON files to allow the execution without recompiling the code.

2.2 Critical Physical Assumptions and Integration Logic

2.2.1 Numerical Implementation and Physical Assumptions

Since the numerical integration method implemented in this project is less sophisticated than high-precision standards for chaotic N-body simulations—such as the **IAS15** integrator [2]—this analysis adopts several essential assumptions to for its proper functioning.

A primary consideration is the **back-extrapolation of initial conditions**. While real-world planetary states result from complex stochastic processes, this simulation initializes all bodies at the $y = 0$ intercept, with x

corresponding to their respective orbital radii. Because the **Beeman** integration scheme requires a previous acceleration state $\mathbf{a}_{t-\Delta t}$ to compute the subsequent velocity, a previous position is calculated as follows:

$$\mathbf{r}_{t-\Delta t} = \mathbf{r}_t - \mathbf{v}_t \Delta t + \frac{1}{2} \mathbf{a}_t \Delta t^2 \quad (2)$$

This allows for the determination of an initial previous acceleration, ensuring the algorithm maintains its high-order accuracy from the first timestep.

Furthermore, to simplify the analysis of numerical stability, we assume that all planets begin with their theoretical circular orbital velocity perpendicular to their initial position vector:

$$v_c = \sqrt{\frac{GM}{r}} \quad (3)$$

By neglecting initial orbital eccentricity, we establish an idealized case where total energy and orbital radius remain theoretically constant. This configuration is critical as it isolates numerical energy drift from the physical energy fluctuations inherent to elliptical orbits, providing a clear baseline for evaluating the long-term performance of the chosen integration schemes.

2.2.2 Simulation Orchestration and Experimental Features

The temporal progression of the system is managed by a centralized `step_forward` method within the `Simulation` class, which enforces a specific sequence of position and velocity updates. For the **Beeman** and **Direct Euler** implementations, the positions are updated first using the current state $(\mathbf{r}_t, \mathbf{v}_t)$, followed by a velocity update once the new acceleration $\mathbf{a}_{t+\Delta t}$ is resolved.

A critical exception exists for the **Euler-Cromer** integration. In this case, the update order reverses: the velocity $\mathbf{v}_{t+\Delta t}$ is calculated first, and this value then determines the new position $\mathbf{r}_{t+\Delta t}$. To preserve the modularity of the update cycle, I encapsulate the velocity calculation directly within the `update_position` method for the `EulerCromer` class.

To facilitate real-time monitoring, I customize the `MultiAnimator` class with an interactive event handler. Pressing the "space" key pauses the integration and triggers an instantaneous printout of the period of the last completed orbit and the time t when it was completed. Furthermore, the simulation identifies planetary alignments by calculating the circular mean of all angular positions via `scipy.stats.circmean`. I define an alignment as a state in which every body falls within a threshold of 10° tolerance of this mean system angle.

3 Results

The simulation results evaluate the dynamical stability of the planetary system in all three implemented algorithms. Conservation of total energy serves as the primary metric for numerical convergence, analyzed in correlation with the evolution of planetary acceleration. Following this stability assessment, the orbital period accuracy and the resulting solar drift are examined over centennial scales. Finally, the detection of planetary alignment events is reported based on the angular configuration of the system.

3.1 Numerical Stability and Energy Conservation

To quantify the long-term reliability of each integrator, the total energy is monitored over a standard 200-year window. Although the simulation is capable of extending significantly beyond this duration, this timeframe provides a sufficient baseline to evaluate the performance of each algorithm. As illustrated in Figure 2, the Beeman scheme maintains a highly stable energy profile with minimal, bounded fluctuations. The Euler-Cromer method similarly remains bounded, but exhibits significantly larger instantaneous oscillatory noise. Conversely, the Direct Euler implementation demonstrates a characteristic non-physical energy gain, with the total energy increasing monotonically over the sampling period.

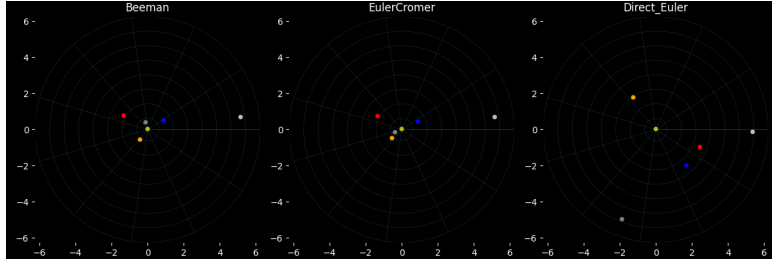


Figure 1: Representative frame of planetary trajectories simulated over 200 years. The Beeman and Euler-Cromer methods maintain stable orbits, while the Direct Euler method demonstrates non-physical orbital expansion.

This divergence is supported by the acceleration plots shown in Figure 3. While the Beeman and Euler-Cromer implementations maintain consistent, periodic acceleration magnitudes, the Direc Euler case shows a precipitous drop as the planets spiral away from the Sun. This physical displacement is captured in Figure 1, where the initial orbital configuration is completely lost as the simulation progresses.

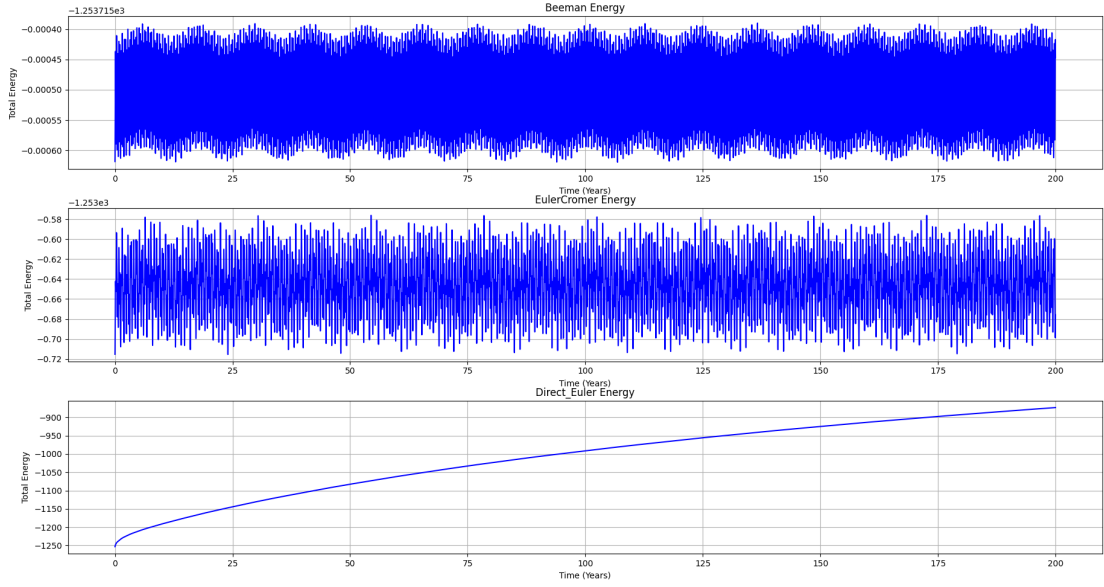


Figure 2: Total energy evolution: Beeman and Euler-Cromer remain bounded, whereas the latter exhibits higher-amplitude oscillatory noise due to lower numerical precision. Conversely, Direct Euler demonstrates non-physical monotonic divergence

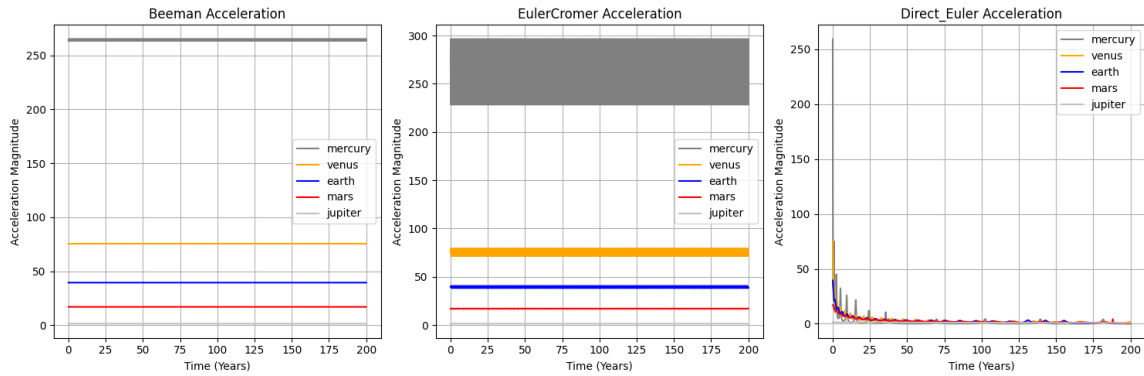


Figure 3: Acceleration profile: Beeman and Euler-Cromer maintain constant acceleration profiles consistent with stable orbits. Conversely, the Direct Euler method shows a rapid decay in acceleration.

3.2 Orbital Accuracy and Solar Dynamics

The physical validity of the integrators is confirmed by comparing simulated orbital periods against observed astronomical data. Table 1 compares the results at $t \approx 12$ years with **NASA** reference values [1]. The Beeman method maintains exceptional accuracy, keeping the Earth's orbital period at exactly 1 year, whereas the Direct Euler method exhibits enormous frequency drift. Furthermore, the simulation captures the dynamic "wobble" of the Sun. Rather than remaining a fixed anchor at the origin, the Sun exhibits a measurable drift trajectory due to the cumulative N-body gravitational pull of the planets.

Planet	NASA Ref (yr)	Beeman (yr)	Euler-Cromer (yr)	Direct Euler (yr)
Mercury	0.241	0.240	0.260	2.520
Venus	0.615	0.610	0.620	8.530
Earth	1.000	1.000	1.000	7.010
Mars	1.881	1.880	1.880	7.400
Jupiter	11.862	11.830	11.810	12.420

Table 1: Comparison of calculated orbital periods (in Earth Years) after $t \approx 12$ years of simulation ($dt = 0.005$). NASA reference values are provided for accuracy comparison.

3.3 Planetary alignment configurations

The simulation successfully identifies planetary alignments throughout the integration period. Using a circular mean threshold of 10° , a notable alignment is detected at $t = 138.69$ years. As shown in Figure 4, all simulated bodies fall within this defined angular proximity at this timestamp. While a 5° limit was initially tested, the 10° tolerance is recommended to capture a more representative range of alignment events. This adjustment is primarily due to the orbital dynamics of Jupiter; its specific orbital period and slower angular velocity relative to the inner planets mean that under a more restrictive threshold, the frequency of detectable alignments is significantly reduced.

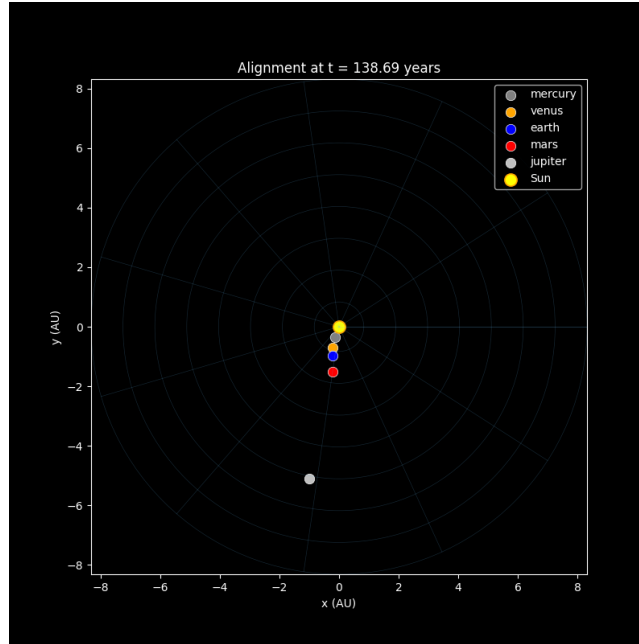


Figure 4: Planets Alignments

4 Discussion

4.1 Comparative Analysis of Numerical Schemes

The varying trajectories demonstrate how critical energy conservation is for orbital stability. The **Direct Euler** method fails because it updates positions using only current velocity, effectively forcing planets along a linear tangent during each timestep (Δt). This results in the cumulative energy growth seen in Figure 2.

Analytically, this follows the inverse-square law ($\propto \frac{1}{r^2}$): as the numerical overshoot increases the orbital radius, the gravitational pull weakens, further accelerating the system’s divergence.

In contrast, the **Beeman** algorithm remains stable by incorporating both current and previous acceleration values. This higher-order correction prevents the numerical drift, allowing the planets to maintain their literature orbital period. The **Euler-Cromer** method serves as a middle ground; although its energy profile shows higher-amplitude noise, it successfully maintains stable orbital radii across the input time duration.

4.2 Physical Realism: Solar Drift and Accuracy

One of the most notable results is the Solar Drift. While many simplified models treat the Sun as a fixed point, the implementation of the Beeman method demonstrates that the Sun moves in a subtle ”wobble.” This is a direct consequence of Newton’s Third Law: as the Sun exerts a gravitational pull on the planets, they exert an equal and opposite force back on the Sun. This enhances the physical realism of the simulation, as all planetary motion is calculated relative to the dynamic position of the Sun.

The Beeman results are considered highly reliable because the orbital periods presented in Table 1 correlate almost exactly with NASA reference values. Although there are minor discrepancies across different simulation runs, the model remains highly predictive over extended temporal scales.

4.3 Alignment and Future Improvements

The detection of this planetary alignment at $t = 138.69$ EY provides additional evidence of the simulation’s long-term stability. Because the Beeman method preserves the system’s orbital integrity, these rare events can be identified with high precision. If the planets drifted even slightly—as observed in the Direct Euler test—this alignment would not be captured. To further improve the model, I would suggest implementing a higher-order integration scheme, such as the **IAS15** integrator [2], to increase planetary motion precision; however, this would require a more complex programming proficiency. Additionally, selecting a 10° tolerance is recommended for future studies to ensure a sufficient range of data. At tighter limits, Jupiter’s position becomes the deciding factor; because it moves so slowly, the chance of all planets falling into a smaller window at the same time is much lower, meaning that many nearly aligned states would be missed.

5 Conclusion

This project has evaluated the dynamical stability of a multi-body solar system, primarily utilizing the Beeman numerical integration scheme and comparing the results against NASA reference data. While the Direct Euler and Euler-Cromer methods are implemented for comparative analysis, the Beeman method proved to be the most robust. By maintaining orbital stability and ensuring total energy conservation, it provides a physically consistent model of celestial motion over extended timescales. This stands in clear contrast to the Direct Euler case, which proved physically incoherent due to lack of energy conservation and resultant orbital drift. The long-term stability of the Beeman integrator facilitated the high-precision identification of planetary alignments, while successfully capturing the Solar Drift, demonstrating that the Sun is a dynamic body responding to the gravitational influence of the orbiting planets.

Future iterations of this work should focus on computational optimization and higher-order numerical integration. Although NumPy arrays provided sufficient performance for this study, implementing gravitational force calculations into a matrix-based architecture would enable full vectorization, significantly improving computational efficiency. Furthermore, attempting the simulation in C++ would allow for a rigorous performance comparison; the reduced execution time in a compiled language would permit the use of much smaller timesteps, thereby enhancing numerical precision beyond the current constraints of Python’s for loops. There is also a significant opportunity to investigate Energy Correction algorithms to reduce numerical noise about the system’s energies. From these I conclude that the stability of the Solar System is as much a matter of numerical precision as it is of gravitational law. The choice of a higher-order integrator is a physical necessity to capture even the smallest detail in a chaotic system.

References

- [1] NASA. Planet compare: Solar system exploration, 2026. Accessed: 2026-04-02.
- [2] Hanno Rein and David S. Spiegel. ias15: a fast, adaptive, high-order integrator for gravitational dynamics, accurate to machine precision over a billion orbits. *Monthly Notices of the Royal Astronomical Society*, 446(2):1424–1437, November 2014.

- [3] Hanno Rein and Daniel Tamayo. A new paradigm for reproducing and analysing n-body simulations of planetary systems. *Monthly Notices of the Royal Astronomical Society*, 467(2):2377–2383, 01 2017.